

Project Design Phase

Solution Architecture

Date	19 July 2026
Team ID	XXXX XXXX XXXX
Project Name	Ucab – Smart Cab Booking Application
Maximum Marks	4 Marks

Architecture Overview

Ucab follows a 3-tier client–server architecture with an additional real-time layer for live tracking, and an MVC pattern on the backend.

Presentation Tier

React.js single-page application (client) — handles booking UI, map view, payment UI, and booking history.

Application Tier

Node.js + Express.js REST API (server) — handles authentication, booking logic, fare calculation, and payment orchestration, structured using the MVC pattern (Models, Views/Routes, Controllers). A Socket.io channel carries real-time location/status events.

Data Tier

MongoDB with Mongoose schemas for Users, Drivers, Bookings, and Payments.

External Services

- Maps/Geocoding API – route calculation and ETA
- Payment gateway API (Razorpay/Stripe) – automatic payment processing

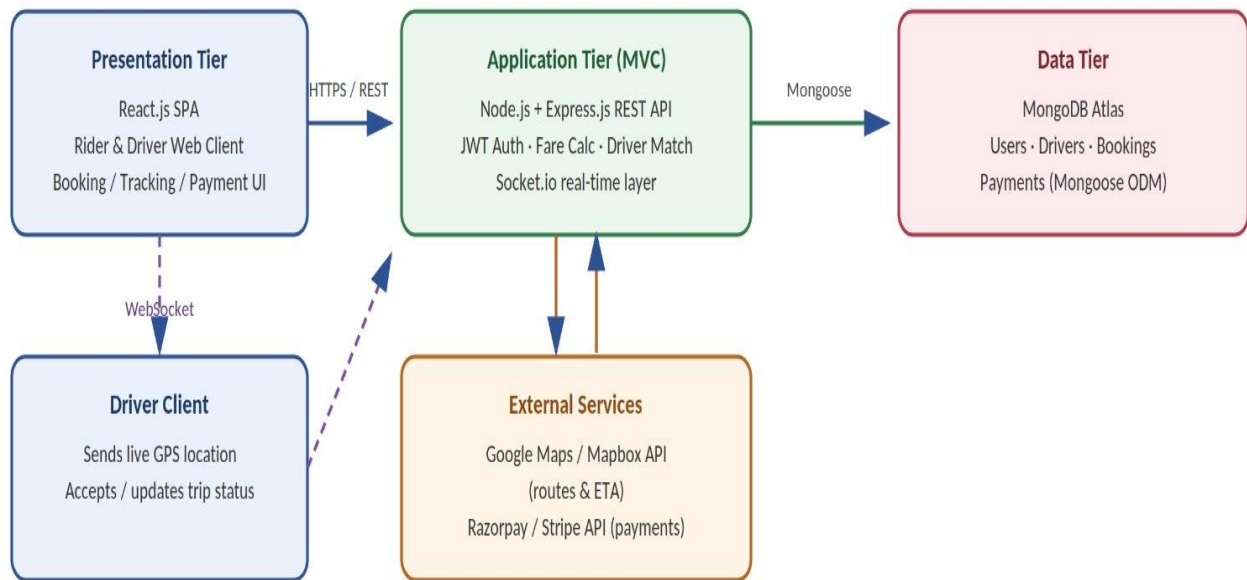
Data & Control Flow

Browser (React) $\rightleftharpoons$ REST API (Express/Node) $\rightleftharpoons$ MongoDB, with a parallel WebSocket channel carrying live cab-location updates: driver client $\rightarrow$ server $\rightarrow$ rider client.

Roles & Responsibilities

Role	Responsibility
Rider	Registers, books rides, tracks cab, pays, views history
Driver	Accepts ride requests, shares live location, updates trip status
Admin	Manages users, drivers, and monitors all bookings
System (Backend)	Validates requests, calculates fares, matches drivers, persists data, broadcasts live updates

Solution Architecture – Ucab



Flow:

Rider Client → REST API → MongoDB Atlas, with a parallel Socket.io channel carrying live location from the Driver Client through the server to the Rider Client. The Application Tier also calls External Services for route/ETA and payment processing.